

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: July 1, 2026
Semester: 1st Semester	Session: Spring 2026
Lab/Project/Assignment #: 9	CLOs to be covered: CLO1
Lab Title: Viva	Teacher Name: Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems.					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
-------	-------	-------	--------

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 9

Lab Goals / Objectives:

- Review and reinforce concepts from Weeks 1-8
- Demonstrate proficiency in loops and conditional statements
- Apply nested loops for complex pattern generation
- Practice dictionary creation and iteration
- Solve problems combining multiple programming concepts
- Prepare for oral examination on programming fundamentals

Equipment Required:

- Computer System with Python 3.x installed
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory:

1. Comprehensive Review of Python Fundamentals

This lab serves as a comprehensive review of all concepts covered in the first eight weeks. Students demonstrate their understanding through practical problem-solving tasks that combine multiple programming concepts.

2. Nested Loops Review

Nested loops are loops placed inside other loops. The inner loop completes all its iterations for each single iteration of the outer loop. Nested loops are essential for working with multi-dimensional data and generating patterns.

```
for i in range(1, 4):  
    for j in range(2, 4):  
        print(i, "and", j)
```

Output:

```
1 and 2  
1 and 3  
2 and 2  
2 and 3  
3 and 2  
3 and 3
```

3. Conditional Statements Review

if-elif-else statements allow programs to make decisions based on conditions. Multiple conditions can be chained using elif (else if) to handle various scenarios.

```
if condition1:  
    # code  
elif condition2:  
    # code  
else:  
    # code
```

4. String Multiplication for Patterns

In Python, strings can be multiplied by integers to repeat them. This is useful for generating patterns efficiently without nested loops:

```
print("*" * 5) # Output: *****
```

5. Dictionary Iteration Review

Dictionaries store key-value pairs. Iterating over a dictionary using the `keys()` method or directly over the dictionary allows access to each key and its corresponding value:

```
for key in dictionary:  
    print(key, dictionary[key])
```

6. Input Validation

Input validation ensures that user-provided data meets expected criteria before processing. This prevents errors and ensures program reliability:

```
if tot <= 0 or got > tot:  
    print("Invalid input!")
```

7. Grade Calculation Logic

Grade calculation involves computing a percentage and mapping it to letter grades using conditional statements. Each grade range corresponds to a specific level of achievement.

8. Combining Concepts

Real-world programming problems often require combining multiple concepts: loops for iteration, conditionals for decision-making, functions for modularity, and data structures for organization.

Lab Work:

Task 1: Number Pairs Using Nested Loops

Write a program that prints all pairs of numbers using nested for loops.

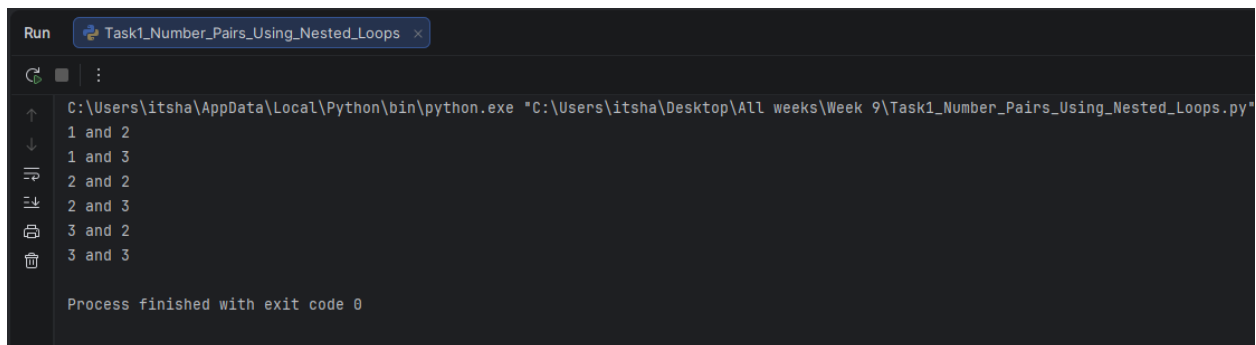
Code Summary:

This task demonstrates nested for loops where the outer loop controls the first number and the inner loop controls the second number, generating all possible combinations.

Key Code Lines:

```
# Number Pairs Using Nested Loops
for i in range(1, 4):
    for j in range(2, 4):
        print(i, "and", j)
```

Output:



```
Run Task1_Number_Pairs_Using_Nested_Loops x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 9\Task1_Number_Pairs_Using_Nested_Loops.py"
1 and 2
1 and 3
2 and 2
2 and 3
3 and 2
3 and 3
Process finished with exit code 0
```

Task 2: Student Grade Calculator Using For Loop

Write a program that calculates grades for multiple students using a for loop and conditional statements.

Code Summary:

This program uses a for loop to process multiple students. For each student, it calculates the percentage and assigns a grade based on conditional checks. Input validation ensures data integrity.

Key Code Lines:

```
# Student Grade Calculator (For Loop)
num = int(input("Enter total number of students: "))
for i in range(num):
    nm = input("Enter student name: ")
    rn = int(input("Enter roll number: "))
    got = int(input("Enter obtained marks: "))
    tot = int(input("Enter total marks: "))
    if tot <= 0 or tot > 300 or got > tot:
```

```

        print("Enter valid numbers")
    else:
        perc = (got / tot) * 100
        print("Percentage:", perc)
        if perc >= 90: print("Grade: A+")
        elif perc >= 85: print("Grade: A-")
        elif perc >= 80: print("Grade: B+")
        elif perc >= 75: print("Grade: B-")
        elif perc >= 70: print("Grade: C+")
        elif perc >= 65: print("Grade: C-")
        else: print("Grade: F")

```

Output:



```

Run Task2_Student_Grade_Calculator_Using_For_Loop x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 9\Task2_Student_Grade_Calculator_Using_For_Loop.py"
Enter total number of students: 3
Enter student name: Hashir Zahid
Enter roll number: 40
Enter obtained marks: 89
Enter total marks: 100
Percentage: 89.0
Grade: A-
Enter student name: najam
Enter roll number: 29
Enter obtained marks: 44
Enter total marks: 100
Percentage: 44.0
Grade: F
Enter student name: usman
Enter roll number: 55
Enter obtained marks: 78
Enter total marks: 100
Percentage: 78.0
Grade: B-

```

Task 3: Items Dictionary Display

Create a dictionary of items and display all key-value pairs using iteration.

Code Summary:

This task demonstrates dictionary creation and iteration. The for loop iterates over dictionary keys, and each key is used to access its corresponding value.

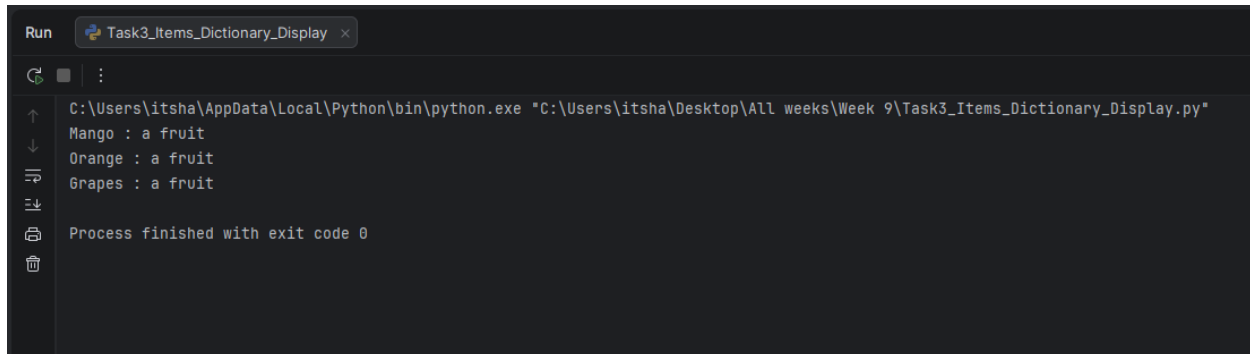
Key Code Lines:

```

# Items Dictionary Display
items = {"Mango": "a fruit", "Orange": "a fruit", "Grapes": "a fruit"}
for k in items:
    print(k, ":", items[k])

```

Output:



```
Run Task3_Items_Dictionary_Display x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 9\Task3_Items_Dictionary_Display.py"
Mango : a fruit
Orange : a fruit
Grapes : a fruit
Process finished with exit code 0
```

Task 4: Right Triangle Star Pattern

Write a program that prints a right triangle star pattern using string multiplication.

Code Summary:

This task uses string multiplication to generate a right triangle pattern efficiently. The number of stars in each row equals the row number.

Key Code Lines:

```
# Right Triangle Star Pattern
r = 7
for i in range(1, r + 1):
    print("*" * i)
```

Output:



```
Run Task4_Right_Triangle_Star_Pattern x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 9\Task4_Right_Triangle_Star_Pattern.py"
*
**
***
****
*****
*****
*****
Process finished with exit code 0
```

Task 5: Student Grade Calculator Using While Loop

Write a program that calculates grades using a while loop instead of a for loop.

Code Summary:

This task demonstrates the same grade calculation logic but using a while loop with a manual counter, showing the flexibility of loop choice based on requirements.

Key Code Lines:

```
# Student Grade Calculator (While Loop)
num = int(input("Enter total number of students: "))
i = 1
while i <= num:
    nm = input("Enter student name: ")
    rn = int(input("Enter roll number: "))
    got = int(input("Enter obtained marks: "))
    tot = int(input("Enter total marks: "))
    if tot <= 0 or tot > 300 or got > tot:
        print("Enter valid numbers")
    else:
        perc = (got / tot) * 100
        print("Percentage:", perc)
        if perc >= 90: print("Grade: A+")
        elif perc >= 85: print("Grade: A-")
        elif perc >= 80: print("Grade: B+")
        elif perc >= 75: print("Grade: B-")
        elif perc >= 70: print("Grade: C+")
        elif perc >= 65: print("Grade: C-")
        else: print("Grade: F")
    i += 1
```

Output:



```
Run Task5_Student_Grade_Calculator_Using_While_Loop x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 9\Task5_Student_Grade_Calculator_Using_While_Loop.py"
Enter total number of students: 1
Enter student name: hashir
Enter roll number: 40
Enter obtained marks: 85
Enter total marks: 100
Percentage: 85.0
Grade: A-

Process finished with exit code 0
```

Task 6: Pyramid Star Pattern

Write a program that prints a pyramid star pattern with proper spacing.

Code Summary:

This task creates a pyramid pattern by printing spaces before stars. The number of spaces decreases while the number of stars increases as we move down the rows.

Key Code Lines:

```
# Pyramid Star Pattern
r = 6
for i in range(1, r):
    print(" " * (r - i), end="")
    print("* " * (2*i - 1))
```

Output:

A screenshot of a Python IDE window titled 'Task6_Pyramid_Star_Pattern'. The command prompt shows the execution of a Python script. The output is a pyramid pattern of stars: a single star on the first line, followed by two stars, three stars, four stars, and five stars on subsequent lines. The message 'Process finished with exit code 0' is displayed at the bottom.

```
Run Task6_Pyramid_Star_Pattern x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 9\Task6_Pyramid_Star_Pattern.py"
    *
   **
  ***
 ****
*****
Process finished with exit code 0
```

Conclusions:

- Successfully reviewed and applied fundamental programming concepts from Weeks 1-8
- Demonstrated proficiency in both for and while loops for iterative problem solving
- Applied nested loops for generating complex patterns and combinations
- Combined conditional statements with loops for decision-making in iterative processes
- Worked with dictionaries for storing and displaying structured data
- Implemented input validation to ensure data integrity
- Gained confidence in solving comprehensive programming problems combining multiple concepts